

CyberScope — Security Posture Analyzer

Software Design Document

Version: 1.0

Author: Mrityunjay Guleria

Date: July 2026

Institution: Dehradun Institute of Technology (DIT)

Program: B.Tech Computer Science — Cybersecurity and Privacy

Table of Contents

1. [Executive Summary](#)
 2. [Objectives](#)
 3. [Scope & Feature Prioritization](#)
 4. [Technology Stack](#)
 5. [System Architecture](#)
 6. [Module Design](#)
 7. [GUI Design](#)
 8. [Database Design](#)
 9. [Security Scoring Model](#)
 10. [Development Timeline](#)
 11. [Testing Strategy](#)
 12. [Security & Legal Considerations](#)
 13. [Future Enhancements](#)
 14. [References](#)
-

1. Executive Summary

CyberScope is a desktop security posture analyzer built in Java/JavaFX that automates the process of assessing an organization's or individual system's security health. It combines three core capabilities:

1. **Network Reconnaissance** — Discovers live hosts, open ports, running services, and operating systems on a target network using Nmap integration
2. **System Hardening Assessment** — Audits the local machine's security configurations (firewall, antivirus, password policies, encryption, user accounts)

3. **Vulnerability Mapping** — Cross-references discovered services and versions against a curated CVE (Common Vulnerabilities and Exposures) database to identify known security risks

These three assessment domains feed into a **weighted security scoring engine** that produces a normalized 0–100 posture score, categorized by risk level. All findings, scores, and remediation recommendations are compiled into a **professional PDF assessment report** suitable for management review.

Why this project exists: Security posture assessment is one of the first steps in any VAPT (Vulnerability Assessment and Penetration Testing) engagement. Commercial tools like Nessus (\$4,990/year), Qualys, and OpenVAS perform this function at enterprise scale. CyberScope demonstrates the engineering principles behind these tools in a focused, educational implementation.

Target users: Security analysts, system administrators, and IT students performing authorized security assessments on networks they own or have explicit permission to test.

2. Objectives

2.1 Technical Objectives

#	Objective	Success Criteria
T1	Integrate Nmap for automated network scanning	Successfully scan a target, parse XML output, display hosts/ports/services
T2	Audit system security configurations	Check ≥ 10 security controls on Windows, ≥ 7 on Linux
T3	Map services to known vulnerabilities	Match service name + version against ≥ 50 curated CVEs
T4	Compute a normalized security posture score	Produce a 0–100 score with category breakdowns
T5	Generate professional PDF reports	Export findings with executive summary, charts, and remediation steps
T6	Build a professional-grade JavaFX interface	Dark-themed, sidebar-navigated, responsive GUI with charts

2.2 Learning Objectives

#	Objective	Relevance to Career
L1	Understand how professional VAPT tools work internally	Direct internship application
L2	Learn to integrate external	Required skill for security

#	Objective	Relevance to Career
	security tools (Nmap) programmatically	engineering roles
L3	Understand CVE/CVSS scoring and vulnerability databases	Core VAPT knowledge
L4	Practice clean architecture and design patterns	Software engineering fundamentals
L5	Learn JavaFX threading model for responsive desktop applications	Concurrency skills valued in interviews
L6	Understand security hardening baselines	CIS Benchmarks, NIST guidelines

2.3 Portfolio Objectives

- Demonstrate practical cybersecurity engineering (not just tool usage)
- Show ability to build production-quality software
- Create a project that generates meaningful conversation in interviews
- Align with CERT Uttarakhand internship experience

3. Scope & Feature Prioritization

3.1 MVP — Version 1.0 (Must Have)

These features are the minimum for a complete, demonstrable project:

Module	Feature	Priority
Dashboard	Security score gauge, risk level indicator, summary cards, recent scans list	P0
Network Scanner	Target input, scan type selection (Quick/Standard/Deep), live progress, results table (hosts/ports/services)	P0
System Hardening	Windows security checks (firewall, Defender, updates, password policy, UAC, BitLocker, SMBv1, RDP, guest account, AutoRun)	P0
Vulnerability Assessment	Service-to-CVE mapping using local curated database, severity classification, remediation	P0

Module	Feature	Priority
	suggestions	
Scoring Engine	4-category weighted score (Ports, Hardening, Vulnerabilities, Configuration), normalized to 0–100	P0
Report Generator	PDF export with executive summary, score breakdown, findings tables, recommendations	P0
Settings	Nmap path configuration, scan defaults, report output directory	P0
Authorization	Consent dialog before any network scan (legal requirement)	P0

3.2 Version 2.0 Features (Nice to Have)

Feature	Description
Linux System Hardening	UFW/iptables, SSH config, SUID auditing, <code>/etc/login.defs</code> , file permissions
Scan History & Trends	Compare scores across sessions, show improvement/degradation over time
NVD API Integration	Live CVE lookups from NIST National Vulnerability Database when internet is available
Scheduled Scans	Configurable periodic scanning with change detection
Custom Check Profiles	User-defined security baselines (e.g., CIS Level 1, PCI-DSS relevant checks)
Export Formats	HTML and CSV export in addition to PDF

3.3 Advanced / Enterprise Features (Future Scope)

Feature	Description
Multi-host Dashboard	Aggregate view across multiple scanned hosts
Active Directory Audit	Domain-level security checks (GPO, Kerberos config)
SIEM Integration	Export findings to Splunk/ELK via Syslog or JSON
REST API	Expose scanning capabilities via API for integration with other tools

Feature	Description
Compliance Mapping	Map findings to CIS Benchmarks, NIST 800-53, ISO 27001 controls

4. Technology Stack

Every technology choice is justified below. No alternatives are listed — each decision is final.

Technology	Version	Purpose	Why This Choice
Java	17 (LTS)	Core language	Long-Term Support, modern features (records, sealed classes, text blocks), enterprise standard. The LTS designation means security patches until 2029+.
JavaFX	21.0.2	GUI framework	Hardware-accelerated rendering, CSS styling support, FXML declarative layouts, built-in chart library. Replaces Swing as the modern Java UI standard.
AtlantaFX	2.0.1	UI theme	Provides pre-built Material Design components (cards, badges, progress indicators) with dark theme support. Eliminates the need to design every component from scratch.
Maven	3.9+	Build tool	Industry-standard dependency management. Declares all libraries in <code>pom.xml</code> — anyone can build the project with <code>mvn package</code> .
SQLite	3.45.1 (via sqlite-jdbc)	Database	Zero-configuration embedded database. The entire database is a single file — no

Technology	Version	Purpose	Why This Choice
			MySQL/PostgreSQL server installation required. Perfect for desktop applications.
Nmap	7.94+ (external)	Network scanner	Industry-standard network reconnaissance tool. Supports service/version detection, OS fingerprinting, and NSE scripting. Outputs structured XML for programmatic consumption.
Apache PDFBox	3.0.1	PDF generation	Pure Java, Apache 2.0 license (no AGPL restrictions), actively maintained. Creates PDF documents programmatically without external dependencies.
Jackson	2.16.1	XML/JSON parsing	Parses Nmap's XML output and CVE JSON data into Java objects. Mature, fast, well-documented.

4.1 Dependencies Not Included (And Why)

Excluded	Reason
iText / OpenPDF	AGPL license requires open-sourcing your entire application or purchasing a commercial license. PDFBox avoids this.
SonarQube	Requires a separate server process. Massively over-complex for a desktop tool.
SpotBugs / PMD / Checkstyle	These are <i>application code</i> analysis tools. CyberScope assesses <i>infrastructure security</i> , not Java source code quality. Different domain entirely.
JFreeChart	JavaFX has built-in chart components (PieChart, BarChart, LineChart) that are sufficient. No need for an external charting library.
Hibernate / JPA	ORM frameworks add complexity without benefit for a simple SQLite schema. Plain JDBC

Excluded	Reason
	with a DAO pattern is cleaner and teaches more.

5. System Architecture

5.1 Architectural Pattern: MVC (Model-View-Controller)

```
graph TB
    subgraph "VIEW Layer (FXML + CSS)"
        V1["Dashboard.fxml"]
        V2["NetworkScan.fxml"]
        V3["SystemAnalysis.fxml"]
        V4["VulnAssessment.fxml"]
        V5["Reports.fxml"]
        V6["Settings.fxml"]
    end

    subgraph "CONTROLLER Layer"
        C0["MainController"]
        C1["DashboardController"]
        C2["NetworkScanController"]
        C3["SystemAnalysisController"]
        C4["VulnAssessmentController"]
        C5["ReportsController"]
        C6["SettingsController"]
    end

    subgraph "SERVICE Layer (Business Logic)"
        S1["NmapExecutor"]
        S2["NmapOutputParser"]
        S3["SystemAnalyzerFactory"]
        S4["WindowsSecurityAnalyzer"]
        S5["VulnerabilityMapper"]
        S6["ScoringEngine"]
        S7["ReportGenerator"]
    end

    subgraph "DATA Layer"
        D1["SQLite Database"]
        D2["ScanRepository"]
        D3["HostRepository"]
        D4["VulnerabilityRepository"]
    end

    subgraph "EXTERNAL"
        E1["Nmap CLI"]
        E2["OS Commands"]
    end

    V1 & V2 & V3 & V4 & V5 & V6 --> C0
    C0 --> C1 & C2 & C3 & C4 & C5 & C6
    C2 --> S1
    S1 --> E1
    E1 -->|"XML Output"| S2
    C3 --> S3
    S3 --> S4
    S4 --> E2
    C4 --> S5
```

```

C1 --> S6
C5 --> S7
S2 & S4 & S5 --> D2 & D3 & D4
D2 & D3 & D4 --> D1
S6 --> D1
S7 --> D1

```

5.2 Data Flow — Complete Scan Lifecycle

```

sequenceDiagram
    participant U as User
    participant UI as JavaFX UI Thread
    participant BG as Background Thread
    participant NM as Nmap Process
    participant SA as System Analyzer
    participant VM as Vulnerability Mapper
    participant SE as Scoring Engine
    participant DB as SQLite
    participant PDF as PDFBox

    U->>UI: Click "Start Full Scan"
    UI->>UI: Show authorization dialog
    U->>UI: Confirm authorization
    UI->>DB: Create ScanSession record
    UI->>BG: Launch scan Task (background thread)

    par Network Scan
        BG->>NM: Execute nmap -sV -O -oX target
        NM-->>BG: XML output file
        BG->>BG: Parse XML → Host/Port/Service objects
        BG->>DB: Store hosts, ports, services
    and System Analysis
        BG->>SA: Run security checks
        SA-->>BG: List of SecurityCheck results
        BG->>DB: Store check results
    end

    BG->>VM: Map services → CVE database
    VM->>DB: Query known_cves table
    DB-->>VM: Matching CVEs
    VM-->>BG: Vulnerability objects
    BG->>DB: Store vulnerabilities

    BG->>SE: Calculate posture score
    SE->>DB: Read all findings
    SE-->>BG: SecurityScore (72/100, category breakdown)
    BG->>DB: Store score breakdown

    BG-->>UI: Platform.runLater() → Update dashboard
    UI->>U: Display results

    opt User clicks "Generate Report"
        UI->>BG: Launch PDF generation task
        BG->>DB: Read all session data
        BG->>PDF: Build PDF document
        PDF-->>BG: PDF file saved
        BG-->>UI: Notify completion
    end
end

```

5.3 Threading Model

[!WARNING] This is the most common failure point in JavaFX applications. The

JavaFX Application Thread manages all UI updates. If you run a 5-minute Nmap scan on this thread, the entire application freezes — buttons become unresponsive, the window shows "Not Responding." This is unacceptable.

Solution: Every long-running operation uses `javafx.concurrent.Task<>`:

```
// CORRECT: Scan runs on background thread
Task<ScanResult> scanTask = new Task<>() {
    @Override
    protected ScanResult call() throws Exception {
        updateMessage("Initializing Nmap scan...");
        updateProgress(0, 100);

        // This runs on a background thread – safe for long operations
        NmapResult networkResult = nmapExecutor.scan(target, scanType);
        updateProgress(40, 100);

        List<SecurityCheck> systemResult = systemAnalyzer.runAllChecks();
        updateProgress(60, 100);

        List<Vulnerability> vulns =
vulnMapper.mapVulnerabilities(networkResult);
        updateProgress(80, 100);

        SecurityScore score = scoringEngine.calculate(networkResult,
systemResult, vulns);
        updateProgress(100, 100);

        return new ScanResult(networkResult, systemResult, vulns, score);
    }
};

// UI updates happen via bindings (thread-safe)
progressBar.progressProperty().bind(scanTask.progressProperty());
statusLabel.textProperty().bind(scanTask.messageProperty());

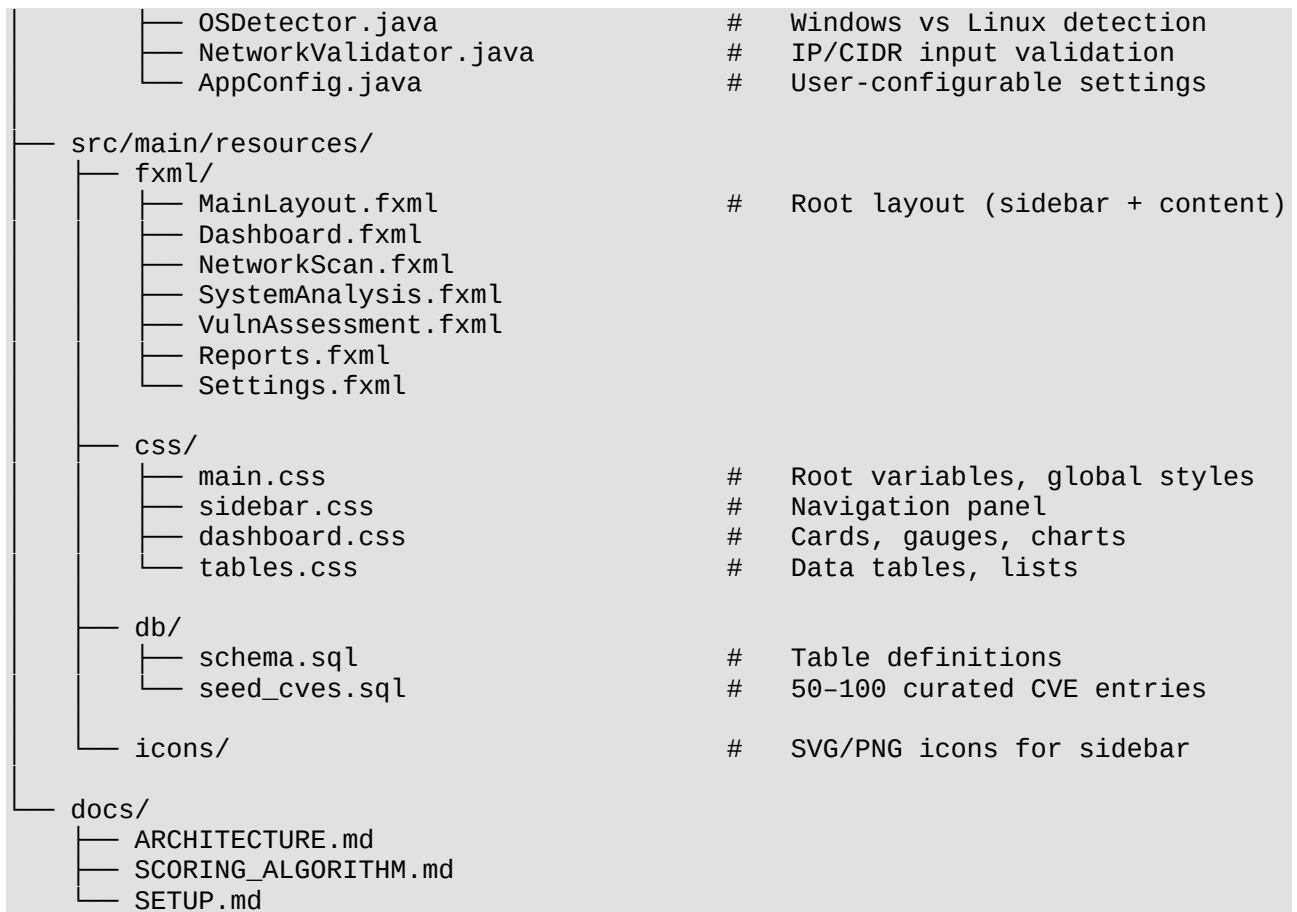
// When complete, update UI on the JavaFX thread
scanTask.setOnSucceeded(event -> {
    ScanResult result = scanTask.getValue();
    dashboardController.updateScore(result.getScore()); // Safe: runs on FX
thread
});

// Launch on background thread
new Thread(scanTask).start();
```

5.4 Project Structure

```
cyberscope/
├── pom.xml
├── src/main/java/com/cyberscope/
│   ├── App.java # JavaFX Application entry point
│   └── MainController.java # Sidebar navigation + content
├── switching
│   └── model/ # Data classes (no logic, no UI)
│       ├── ScanSession.java # id, type, target, timestamps,
├── status
│   └── Host.java # ip, hostname, os, accuracy
```

	Port.java	#	number, protocol, state
	Service.java	#	name, product, version
	Vulnerability.java	#	cveId, cvss, severity,
description			
	SecurityCheck.java	#	category, name, status, details
	SecurityScore.java	#	total, categoryScores,
riskLevel			
	enums/		
	Severity.java	#	LOW, MEDIUM, HIGH, CRITICAL
	CheckStatus.java	#	PASS, FAIL, WARNING, INFO,
ERROR			
	RiskLevel.java	#	EXCELLENT, GOOD, MODERATE,
HIGH, CRITICAL			
	ScanType.java	#	QUICK, STANDARD, DEEP, UDP
	service/		
	scanner/		
	NmapCommandBuilder.java	#	Builds safe Nmap command
strings			
	NmapExecutor.java	#	Spawns Nmap process, manages
lifecycle			
	NmapOutputParser.java	#	Parses XML → Host/Port/Service
objects			
	analyzer/		
	SystemAnalyzer.java	#	Interface: runAllChecks()
	SystemAnalyzerFactory.java	#	Returns Win or Linux analyzer
	WindowsSecurityAnalyzer.java	#	10+ Windows security checks
	LinuxSecurityAnalyzer.java	#	7+ Linux security checks (v2)
	vulnerability/		
	VulnerabilityMapper.java	#	Maps services → CVEs
	VersionComparator.java	#	Semantic version range matching
	CVEDatabase.java	#	Local SQLite CVE lookups
	scoring/		
	ScoringEngine.java	#	Orchestrates score calculation
	PortScoreCalculator.java	#	Scores network exposure
	HardeningScoreCalculator.java	#	Scores system hardening
	VulnScoreCalculator.java	#	Scores vulnerability findings
	ConfigScoreCalculator.java	#	Scores security configuration
	report/		
	ReportGenerator.java	#	Orchestrates full report
	PDFBuilder.java	#	Low-level PDFBox operations
	controller/		
	DashboardController.java		
	NetworkScanController.java		
	SystemAnalysisController.java		
	VulnAssessmentController.java		
	ReportsController.java		
	SettingsController.java		
	repository/		
	DatabaseManager.java	#	Data Access Objects (DAO pattern)
		#	Connection pool, schema
initialization			
	ScanSessionRepository.java		
	HostRepository.java		
	VulnerabilityRepository.java		
	SecurityCheckRepository.java		
	util/		
	ProcessExecutor.java	#	Generic OS command execution



6. Module Design

6.1 Module 1: Network Scanner

Purpose: Discover live hosts, open ports, running services, and operating systems on a target network.

How it works:

```
flowchart LR
    A["User enters<br>target IP/range"] --> B["NetworkValidator<br>validates input"]
    B --> C["NmapCommandBuilder<br>constructs command"]
    C --> D["NmapExecutor runs<br>Nmap via ProcessBuilder"]
    D --> E["Nmap produces<br>XML output file"]
    E --> F["NmapOutputParser<br>parses XML"]
    F --> G["Host, Port, Service<br>objects created"]
    G --> H["Stored in SQLite"]
    G --> I["Displayed in UI table"]
```

Nmap Scan Presets:

Preset	Command Flags	Ports Scanned	Time Estimate	Use Case
Quick	-T4 -F --top-ports 100	Top 100	~30 seconds	Rapid overview

Preset	Command Flags	Ports Scanned	Time Estimate	Use Case
Standard	-sV --top-ports 1000	Top 1000	~3–5 minutes	Default assessment
Deep	-sV -O -sC -p-	All 65,535	~15–30 minutes	Thorough assessment
UDP	-sU --top-ports 50	Top 50 UDP	~5–10 minutes	DNS, SNMP, DHCP discovery

Input Validation (critical for security):

- IPv4 address: `192.168.1.1` → validated against `^(\d{1,3}\.){3}\d{1,3}$` with octet range check (0–255)
- CIDR range: `192.168.1.0/24` → validated subnet mask (8–32)
- Hostname: `server.local` → validated against RFC 1123
- **Reject:** Any input containing shell metacharacters (`;`, `|`, `&`, ```, `$`) to prevent command injection

Error Handling:

Error Condition	Detection	User Response
Nmap not installed	<code>IOException</code> from <code>ProcessBuilder</code>	Show setup instructions dialog
Nmap not in PATH	Non-zero exit code	Prompt to set path in Settings
Target unreachable	Nmap output shows 0 hosts up	"No hosts discovered. Verify target and network connectivity."
Permission denied	Nmap error output contains "requires root"	"Some scan types require administrator privileges."
Scan timeout	Process exceeds configurable timeout (default: 30 min)	"Scan timed out. Try a smaller target range or Quick scan."

6.2 Module 2: System Hardening Analyzer

Purpose: Audit the security configuration of the local machine against established hardening baselines.

Design Pattern: Factory + Strategy

```
// Interface – both Windows and Linux analyzers implement this
public interface SystemAnalyzer {
    List<SecurityCheck> runAllChecks();
    List<SecurityCheck> runCheck(String checkName);
    List<String> getAvailableChecks();
}
```

```
// Factory – auto-detects OS and returns the right implementation
public class SystemAnalyzerFactory {
    public static SystemAnalyzer create() {
        if (OSDetector.isWindows()) {
            return new WindowsSecurityAnalyzer();
        } else if (OSDetector.isLinux()) {
            return new LinuxSecurityAnalyzer();
        }
        throw new UnsupportedOperationException("OS not supported");
    }
}
```

Windows Security Checks (MVP):

#	Check Name	Command / Method	Pass Condition	Severity if Fail
W1	Firewall Status	netsh advfirewall show allprofiles state	All profiles (Domain, Private, Public) = ON	Critical
W2	Windows Defender Real-Time Protection	powershell Get-MpComputerStatus	RealTimeProtectionEnabled = True	Critical
W3	Windows Defender Definitions	powershell Get-MpComputerStatus	Definitions updated within 7 days	● Medium
W4	Windows Update Recency	powershell Get-HotFix Sort InstalledOn -Desc	Last hotfix within 30 days	● High
W5	Password Minimum Length	net accounts	Min password length ≥ 8 characters	● High
W6	Account Lockout Threshold	net accounts	Lockout threshold ≤ 5 attempts	● Medium
W7	UAC Enabled	Registry: EnableLUA = 1	UAC enabled	● High
W8	Guest Account Disabled	net user Guest	Account is disabled	● Medium
W9	SMBv1 Disabled	powershell Get-SmbServerConfiguration	EnableSMB1Protocol = False	● High
W10	Remote Desktop Status	Registry: fDenyTSConne	RDP disabled (unless explicitly	● Medium

#	Check Name	Command / Method	Pass Condition	Severity if Fail
		ctions	needed)	
W11	BitLocker Encryption	manage-bde -status	System drive encrypted	● High
W12	AutoRun Disabled	Registry: NoDriveTypeAutoRun	AutoRun disabled	⦿ Medium

How each check works internally:

```
// Example: Checking firewall status on Windows
public SecurityCheck checkFirewall() {
    try {
        // Execute the netsh command
        ProcessResult result = ProcessExecutor.run(
            "netsh", "advfirewall", "show", "allprofiles", "state"
        );

        String output = result.getStdout();
        // Parse output – look for "State ON"
        boolean domainOn = output.contains("Domain Profile") &&
            containsStateOn(output, "Domain");
        boolean privateOn = containsStateOn(output, "Private");
        boolean publicOn = containsStateOn(output, "Public");

        if (domainOn && privateOn && publicOn) {
            return new SecurityCheck("FIREWALL", "Windows Firewall",
                CheckStatus.PASS, "All firewall profiles are enabled.",
                null);
        } else {
            String details = buildDisabledProfiles(domainOn, privateOn,
                publicOn);
            return new SecurityCheck("FIREWALL", "Windows Firewall",
                CheckStatus.FAIL, details,
                "Enable all firewall profiles: netsh advfirewall set allprofiles
                state on");
        }
    } catch (IOException e) {
        return new SecurityCheck("FIREWALL", "Windows Firewall",
            CheckStatus.ERROR, "Unable to query firewall status: " +
            e.getMessage(),
            null);
    }
}
```

6.3 Module 3: Vulnerability Assessment

Purpose: Cross-reference discovered services and their versions against known CVEs to identify security risks.

How CVE mapping works:

```
flowchart TD
    A["Discovered Service:<br>Apache httpd 2.4.41"] --> B["Extract:<br>name='apache'<br>version='2.4.41'"]
```

```

B --> C["Query known_cves table:<br>WHERE service_name = 'apache'<br>AND
'2.4.41' BETWEEN<br>min_version AND max_version"]
C --> D{"Matches found?"}
D -->|Yes| E["CVE-2021-44790<br>CVSS: 9.8, Critical<br><br>CVE-2021-
41773<br>CVSS: 7.5, High"]
D -->|No| F["No known CVEs<br>for this version"]
E --> G["Create Vulnerability<br>objects with remediation"]
F --> G
G --> H["Store in database<br>+ display in UI"]

```

Version Comparison Logic:

Matching a service version to a CVE's affected version range requires **semantic version comparison**. For example, CVE-2021-44790 affects Apache 2.4.0 through 2.4.51. We need to determine that 2.4.41 falls within this range.

```

// Semantic version comparison
// "2.4.41" → [2, 4, 41]
// Compare component by component: 2.4.41 >= 2.4.0 AND 2.4.41 <= 2.4.51 → MATCH
public class VersionComparator {
    public static int compare(String v1, String v2) {
        String[] parts1 = v1.split("\\.");
        String[] parts2 = v2.split("\\.");
        int length = Math.max(parts1.length, parts2.length);
        for (int i = 0; i < length; i++) {
            int p1 = i < parts1.length ? parseNumeric(parts1[i]) : 0;
            int p2 = i < parts2.length ? parseNumeric(parts2[i]) : 0;
            if (p1 != p2) return Integer.compare(p1, p2);
        }
        return 0;
    }

    public static boolean isInRange(String version, String minVersion, String
maxVersion) {
        return compare(version, minVersion) >= 0 && compare(version, maxVersion)
<= 0;
    }
}

```

Curated CVE Seed Data (50–100 entries covering common services):

Service	Example CVEs	CVSS Range	Why Included
Apache HTTP Server	CVE-2021-44790, CVE-2021-41773, CVE-2019-0211	7.5–9.8	Most common web server
OpenSSH	CVE-2023-38408, CVE-2021-41617	6.5–9.8	Present on nearly every Linux host
MySQL / MariaDB	CVE-2023-21980, CVE-2022-21367	5.5–8.8	Common database
nginx	CVE-2022-41741, CVE-2021-23017	7.4–9.4	Second most common web server
vsftpd / ProFTPD	CVE-2021-3618, CVE-2019-12815	5.9–9.8	FTP still widely deployed
Samba (SMB)	CVE-2017-7494, CVE-2021-44142	7.5–9.8	Windows file sharing, WannaCry vector

Service	Example CVEs	CVSS Range	Why Included
Apache Tomcat	CVE-2020-1938 (GhostCat)	9.8	Java application server
PostgreSQL	CVE-2023-39417	8.8	Open-source database
Redis	CVE-2022-0543	10.0	In-memory database, often misconfigured
Docker Engine	CVE-2024-21626	8.6	Container runtime

6.4 Module 4: Security Scoring Engine

(Detailed algorithm in Section 9)

Purpose: Aggregate all findings into a single, normalized security posture score that communicates overall risk level.

Design: Four independent calculators, each producing a 0–100 score, combined with documented weights.

```
public class ScoringEngine {
    private static final double WEIGHT_PORTS = 0.20;
    private static final double WEIGHT_HARDENING = 0.25;
    private static final double WEIGHT_VULNS = 0.35;
    private static final double WEIGHT_CONFIG = 0.20;

    public SecurityScore calculate(ScanSession session) {
        double portScore = portCalculator.calculate(session);
        double hardenScore = hardeningCalculator.calculate(session);
        double vulnScore = vulnCalculator.calculate(session);
        double configScore = configCalculator.calculate(session);

        double total = (portScore * WEIGHT_PORTS)
            + (hardenScore * WEIGHT_HARDENING)
            + (vulnScore * WEIGHT_VULNS)
            + (configScore * WEIGHT_CONFIG);

        total = Math.max(0, Math.min(100, total)); // Clamp to [0, 100]

        RiskLevel risk = RiskLevel.fromScore(total);
        return new SecurityScore(total, portScore, hardenScore,
                                vulnScore, configScore, risk);
    }
}
```

6.5 Module 5: Report Generator

Purpose: Produce a professional PDF security assessment report.

Report Structure:

Section	Content	Page(s)
Cover Page	Title, date, target, assessor	1

Section	Content	Page(s)
	name, organization	
Executive Summary	Overall score, risk level, key findings count, top 3 recommendations	1
Score Breakdown	Category scores with bar chart, risk level explanation	1
Network Scan Results	Table: Host IP, Hostname, OS, Open Ports, Services	1–2
System Hardening Findings	Table: Check Name, Status (Pass/Fail/Warning), Details	1–2
Vulnerability Details	Table: CVE ID, Service, CVSS, Severity, Description, Remediation	1–3
Recommendations	Prioritized remediation steps, ordered by severity	1
Appendix	Scan configuration, timestamps, tool versions	1

PDF Generation with PDFBox:

```
public class PDFBuilder {
    private PDDocument document;
    private PDPPage currentPage;
    private PDPPageContentStream content;
    private float yPosition;

    // High-level methods that abstract PDFBox complexity
    public void addTitle(String text) { /* font size 24, bold, centered */ }
    public void addHeading(String text) { /* font size 16, bold, left-aligned */ }
}

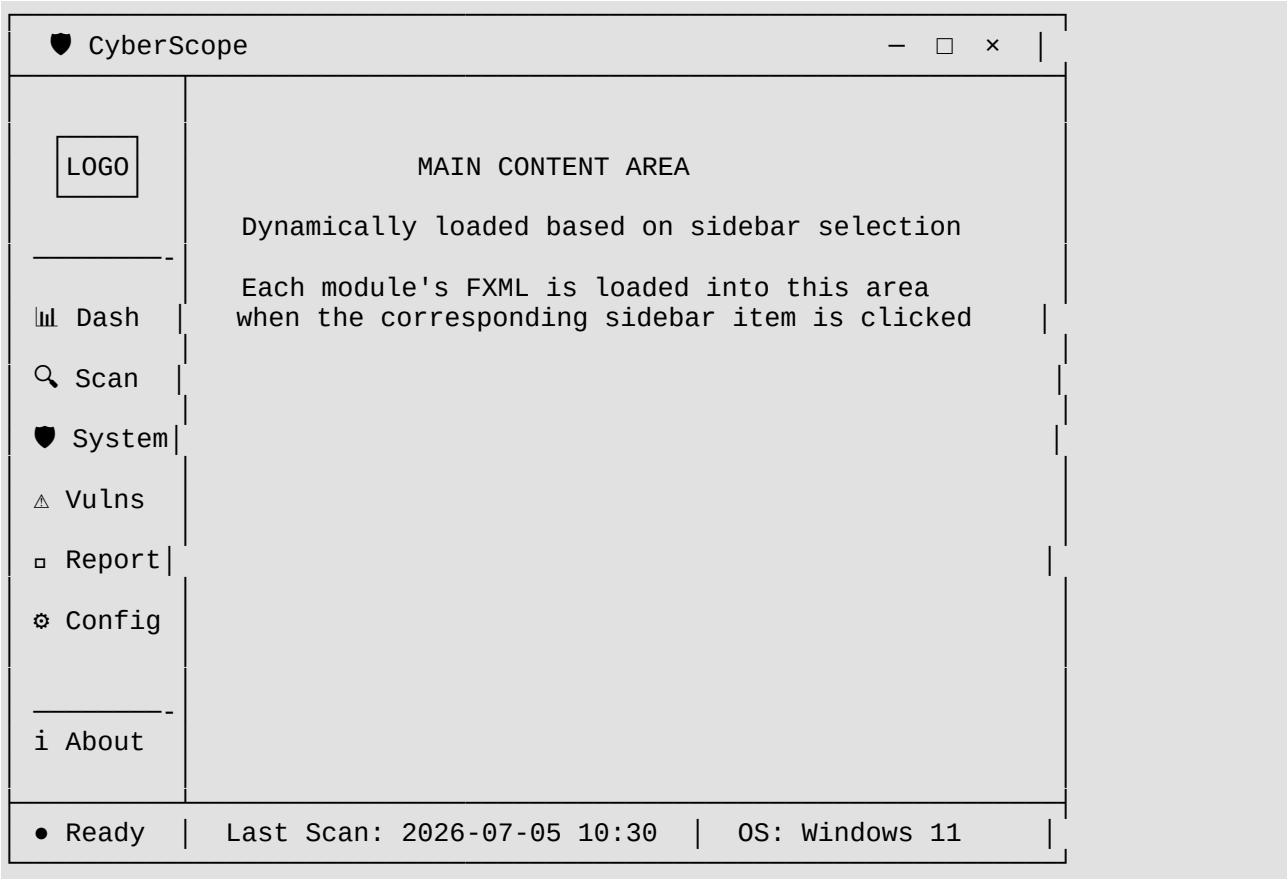
    public void addParagraph(String text) { /* font size 11, regular, word-
wrapped */ }
    public void addTable(String[] headers, String[][] data) { /* bordered table
*/ }
    public void addChart(BufferedImage chartImage) { /* embed chart as image
*/ }
    public void addPageBreak() { /* start new page */ }
    public void save(String filePath) { /* write to disk */ }
}
```

Chart Generation for Reports:

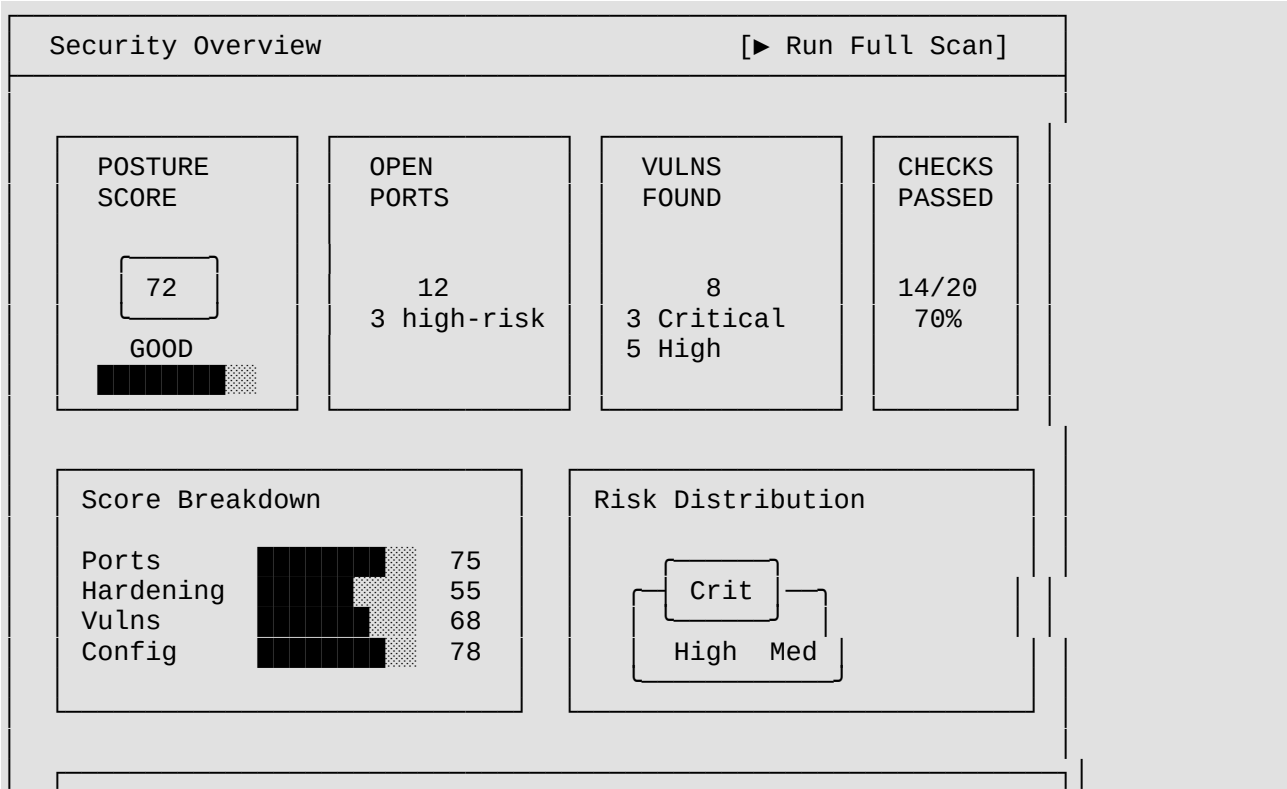
Charts are generated using JavaFX's built-in chart components, rendered to a `BufferedImage` via `WritableImage` snapshot, then embedded in the PDF. This approach reuses the same chart code from the dashboard.

7. GUI Design

7.1 Application Layout



7.2 Dashboard Layout



Recent Scans				
Date	Type	Target	Score	Status
2026-07-05	Full	192.168.1.0/24	72	Complete
2026-07-03	Network	10.0.0.0/24	81	Complete
2026-07-01	System	localhost	65	Complete

7.3 Network Scan Page Layout

Network Scanner

Target: [192.168.1.0/24] Scan Type: [Standard ▼]

[▶ Start Scan] [■ Stop]

67% Scanning 192.168.1.45...

Discovered Hosts (7)

IP Address	Hostname	OS	Ports Open
192.168.1.1	gateway.local	Linux 5.x	3
192.168.1.10	webserver	Ubuntu 22.04	5
192.168.1.20	dbserver	CentOS 8	2
192.168.1.50	workstation-1	Windows 11	4

Port Details for: 192.168.1.10 (webserver)

Port	Proto	State	Service	Product	Version
22	TCP	open	ssh	OpenSSH	8.9p1
80	TCP	open	http	Apache	2.4.52
443	TCP	open	https	Apache	2.4.52
3306	TCP	open	mysql	MySQL	8.0.31
8080	TCP	open	http	Tomcat	9.0.65

7.4 Color System

Primary Palette (Dark Theme):

Token	Hex	RGB	Usage
--bg-primary	#0F1923	15, 25, 35	Main application background
--bg-secondary	#0A1128	10, 17, 40	Sidebar background
--bg-card	#1A2332	26, 35, 50	Content cards
--bg-card-hover	#243447	36, 52, 71	Card hover state
--border	#2A3A4E	42, 58, 78	Card borders, dividers

Accent Colors:

Token	Hex	Usage
--accent-primary	#00D4FF	Active sidebar item, primary buttons, links
--success	#00E676	Pass status, excellent/good scores
--warning	#FFB300	Warning status, moderate scores
--danger	#FF5252	Fail status, high/critical risk
--info	#448AFF	Informational elements

Text Colors:

Token	Hex	Usage
--text-primary	#ECEFF1	Headings, primary content
--text-secondary	#90A4AE	Descriptions, metadata, labels
--text-muted	#546E7A	Disabled items, placeholders

CSS Implementation:

```
/* main.css – Root variables */
.root {
  -fx-base: #0F1923;
  -fx-background: #0F1923;
  -fx-control-inner-background: #1A2332;
  -fx-accent: #00D4FF;
  -fx-focus-color: #00D4FF;

  /* Custom properties */
  --bg-primary: #0F1923;
  --bg-sidebar: #0A1128;
  --bg-card: #1A2332;
  --accent: #00D4FF;
  --success: #00E676;
  --warning: #FFB300;
  --danger: #FF5252;
  --text-primary: #ECEFF1;
  --text-secondary: #90A4AE;
}

/* Sidebar styling */
.sidebar {
  -fx-background-color: #0A1128;
  -fx-pref-width: 220px;
  -fx-padding: 0;
}

.sidebar-item {
  -fx-background-color: transparent;
  -fx-text-fill: #90A4AE;
  -fx-padding: 12 20;
  -fx-cursor: hand;
}
```

```

    -fx-font-size: 14px;
}

.sidebar-item:hover {
    -fx-background-color: #1A2332;
    -fx-text-fill: #ECEFF1;
}

.sidebar-item.active {
    -fx-background-color: rgba(0, 212, 255, 0.1);
    -fx-text-fill: #00D4FF;
    -fx-border-color: transparent transparent transparent #00D4FF;
    -fx-border-width: 0 0 0 3;
}

/* Card component */
.stat-card {
    -fx-background-color: #1A2332;
    -fx-background-radius: 12;
    -fx-border-color: #2A3A4E;
    -fx-border-radius: 12;
    -fx-padding: 20;
    -fx-effect: dropshadow(gaussian, rgba(0,0,0,0.3), 10, 0, 0, 4);
}

.stat-card:hover {
    -fx-background-color: #243447;
    -fx-border-color: #00D4FF;
}

```

8. Database Design

8.1 Entity-Relationship Diagram

```

erDiagram
    SCAN_SESSIONS ||--o{ HOSTS : contains
    SCAN_SESSIONS ||--o{ SECURITY_CHECKS : produces
    SCAN_SESSIONS ||--o{ SCORE_BREAKDOWN : has
    HOSTS ||--o{ PORTS : has
    HOSTS ||--o{ VULNERABILITIES : affected_by
    PORTS ||--o{ VULNERABILITIES : associated_with
    KNOWN_CVES ||--o{ VULNERABILITIES : referenced_by

    SCAN_SESSIONS {
        int id PK
        string scan_type
        string target
        timestamp started_at
        timestamp completed_at
        string status
        real overall_score
    }

    HOSTS {
        int id PK
        int session_id FK
        string ip_address
        string hostname
        string os_name
        int os_accuracy
    }

```

```

}

PORTS {
    int id PK
    int host_id FK
    int port_number
    string protocol
    string state
    string service_name
    string service_version
    string product
}

VULNERABILITIES {
    int id PK
    int session_id FK
    int host_id FK
    int port_id FK
    string cve_id
    real cvss_score
    string severity
    string title
    string description
    string remediation
}

SECURITY_CHECKS {
    int id PK
    int session_id FK
    string category
    string check_name
    string status
    string details
    string remediation
}

KNOWN_CVES {
    string cve_id PK
    string service_name
    string min_version
    string max_version
    real cvss_score
    string severity
    string description
    string remediation
    string reference_url
}

SCORE_BREAKDOWN {
    int id PK
    int session_id FK
    string category
    real score
    real weight
    string deduction_details
}

```

8.2 SQL Schema

```

CREATE TABLE scan_sessions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    scan_type     TEXT NOT NULL CHECK(scan_type IN
('NETWORK', 'SYSTEM', 'FULL')),

```

```

        target          TEXT,
        started_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        completed_at    TIMESTAMP,
        status          TEXT DEFAULT 'RUNNING' CHECK(status IN
('RUNNING', 'COMPLETED', 'FAILED', 'CANCELLED')),
        overall_score   REAL CHECK(overall_score >= 0 AND overall_score <= 100)
    );

CREATE TABLE hosts (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id         INTEGER NOT NULL REFERENCES scan_sessions(id) ON DELETE
CASCADE,
    ip_address         TEXT NOT NULL,
    hostname           TEXT,
    os_name            TEXT,
    os_accuracy        INTEGER CHECK(os_accuracy >= 0 AND os_accuracy <= 100)
);
CREATE INDEX idx_hosts_session ON hosts(session_id);

CREATE TABLE ports (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    host_id            INTEGER NOT NULL REFERENCES hosts(id) ON DELETE CASCADE,
    port_number        INTEGER NOT NULL CHECK(port_number >= 0 AND port_number <=
65535),
    protocol           TEXT NOT NULL CHECK(protocol IN ('tcp', 'udp')),
    state              TEXT NOT NULL,
    service_name       TEXT,
    service_version    TEXT,
    product            TEXT
);
CREATE INDEX idx_ports_host ON ports(host_id);

CREATE TABLE vulnerabilities (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id         INTEGER NOT NULL REFERENCES scan_sessions(id) ON DELETE
CASCADE,
    host_id            INTEGER REFERENCES hosts(id),
    port_id            INTEGER REFERENCES ports(id),
    cve_id             TEXT,
    cvss_score         REAL CHECK(cvss_score >= 0 AND cvss_score <= 10),
    severity           TEXT NOT NULL CHECK(severity IN
('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    title              TEXT NOT NULL,
    description        TEXT,
    remediation        TEXT,
    reference_url      TEXT
);
CREATE INDEX idx_vulns_session ON vulnerabilities(session_id);
CREATE INDEX idx_vulns_severity ON vulnerabilities(severity);

CREATE TABLE security_checks (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id         INTEGER NOT NULL REFERENCES scan_sessions(id) ON DELETE
CASCADE,
    category           TEXT NOT NULL,
    check_name         TEXT NOT NULL,
    status            TEXT NOT NULL CHECK(status IN
('PASS', 'FAIL', 'WARNING', 'INFO', 'ERROR')),
    details            TEXT,
    remediation        TEXT
);
CREATE INDEX idx_checks_session ON security_checks(session_id);

CREATE TABLE known_cves (

```

```

    cve_id          TEXT PRIMARY KEY,
    service_name    TEXT NOT NULL,
    min_version     TEXT,
    max_version     TEXT,
    cvss_score      REAL NOT NULL CHECK(cvss_score >= 0 AND cvss_score <= 10),
    severity        TEXT NOT NULL CHECK(severity IN
('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
    description     TEXT,
    remediation     TEXT,
    reference_url   TEXT
);
CREATE INDEX idx_cves_service ON known_cves(service_name);

CREATE TABLE score_breakdown (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id      INTEGER NOT NULL REFERENCES scan_sessions(id) ON DELETE
CASCADE,
    category        TEXT NOT NULL CHECK(category IN
('PORTS', 'HARDENING', 'VULNS', 'CONFIG')),
    score           REAL NOT NULL CHECK(score >= 0 AND score <= 100),
    weight          REAL NOT NULL,
    deduction_details TEXT -- JSON array of applied deductions
);
CREATE INDEX idx_scores_session ON score_breakdown(session_id);

```

9. Security Scoring Model

9.1 Design Philosophy

The scoring model is inspired by the **NIST Cybersecurity Framework (CSF)** approach of assessing multiple security domains independently, then aggregating into an overall posture. Each domain is scored on a normalized 0–100 scale before weighting, which avoids the mathematical error of adding incompatible units.

9.2 Scoring Formula

$$\text{Total Score} = \sum_{i=1}^4 (S_i \times W_i)$$

Where:

Symbol	Category	Weight (\$W_i\$)	Justification
\$S_1\$	Port Security	0.20 (20%)	Open ports represent attack surface but are not inherently dangerous if services are patched
\$S_2\$	System Hardening	0.25 (25%)	Disabled defenses (firewall, AV, updates) are the #1 enabler of successful attacks
\$S_3\$	Vulnerability Exposure	0.35 (35%)	Known CVEs represent the highest immediate risk — they have

Symbol	Category	Weight (\$W_i\$)	Justification
			published exploits
\$S_4\$	Security Configuration	0.20 (20%)	Weak configurations (default creds, legacy protocols) create exploitable conditions

Why these weights? Vulnerabilities receive the highest weight because they represent *known, exploitable* risks — a CVE-2021-44790 (CVSS 9.8) in Apache can be exploited with a publicly available script. Hardening is second because disabled defenses (no firewall, no AV) dramatically amplify the impact of any attack. Ports and configurations are lower because they represent *potential* risk rather than *confirmed* risk.

9.3 Category Score Calculations

Each category starts at **100** (perfect) and receives **deductions** based on findings. The score is **floored at 0** (never negative).

Port Security Score (\$S_1\$)

$$S_1 = \max(0, 100 - \sum \text{deductions})$$

Finding	Deduction	Reasoning
Each open port (general)	−3	Increases attack surface
Dangerous port open (21, 23, 135, 139, 445, 1433, 3389, 5900)	−8	Historically targeted by attackers, frequently exploited
Service with no version detected	−2	Cannot assess vulnerability without version info
More than 20 open ports on a single host	−10 (bonus)	Excessive exposure indicates poor network segmentation

System Hardening Score (\$S_2\$)

$$S_2 = \max(0, 100 - \sum \text{deductions})$$

Finding	Deduction	Reasoning
Firewall disabled	−20	First line of network defense
Antivirus disabled/missing	−15	No malware protection
OS updates >30 days old	−15	Missing security patches
Password min length < 8	−10	Enables brute-force attacks
No account lockout policy	−10	Allows unlimited login attempts
UAC disabled	−5	Removes privilege escalation barrier
Guest account enabled	−5	Unnecessary access vector

Finding	Deduction	Reasoning
SMBv1 enabled	−10	WannaCry/EternalBlue attack vector
RDP exposed	−5	Remote access target (BlueKeep)
BitLocker not enabled	−5	No disk encryption

Vulnerability Score (\$S_3\$)

$$S_3 = \max(0, 100 - \sum_j D(\text{cvss}_j))$$

CVE Severity	CVSS Range	Deduction per CVE (\$D\$)	Reasoning
Critical	9.0–10.0	−15	Remotely exploitable, severe impact, published exploits likely
High	7.0–8.9	−10	Significant risk, exploitation probable
Medium	4.0–6.9	−5	Moderate risk, may require specific conditions
Low	0.1–3.9	−2	Minor risk, limited impact

Configuration Score (\$S_4\$)

$$S_4 = \max(0, 100 - \sum \text{deductions})$$

Finding	Deduction	Reasoning
Unencrypted protocol in use (FTP/21, Telnet/23, HTTP/80 for admin panels)	−8 each	Data transmitted in cleartext, interceptable
Legacy protocol enabled (SMBv1, SSLv3, TLS 1.0)	−10 each	Known vulnerable, should be deprecated
Unnecessary services running (e.g., print spooler on a server)	−3 each	Reduces attack surface
AutoRun enabled	−5	USB-based malware vector

9.4 Risk Level Classification

Score Range	Risk Level	Color Code	Interpretation
90–100	● Excellent	#00E676	Strong security posture. Minor improvements

Score Range	Risk Level	Color Code	Interpretation
			only.
70–89	Good	#448AFF	Solid baseline. Some areas need attention.
50–69	● Moderate	#FFB300	Significant gaps exist. Action required.
30–49	● High Risk	#FF9100	Serious deficiencies. Immediate remediation needed.
0–29	Critical	#FF5252	Severely insecure. Urgent intervention required.

9.5 Example Calculation

Scenario: A scan discovers:

- 8 open ports (including port 21/FTP and port 3389/RDP) → Port deductions
- Firewall enabled, Defender enabled, but updates 45 days old and no lockout policy → Hardening deductions
- 1 Critical CVE (CVSS 9.8) and 2 High CVEs (CVSS 7.5, 8.1) → Vuln deductions
- FTP (cleartext) and AutoRun enabled → Config deductions

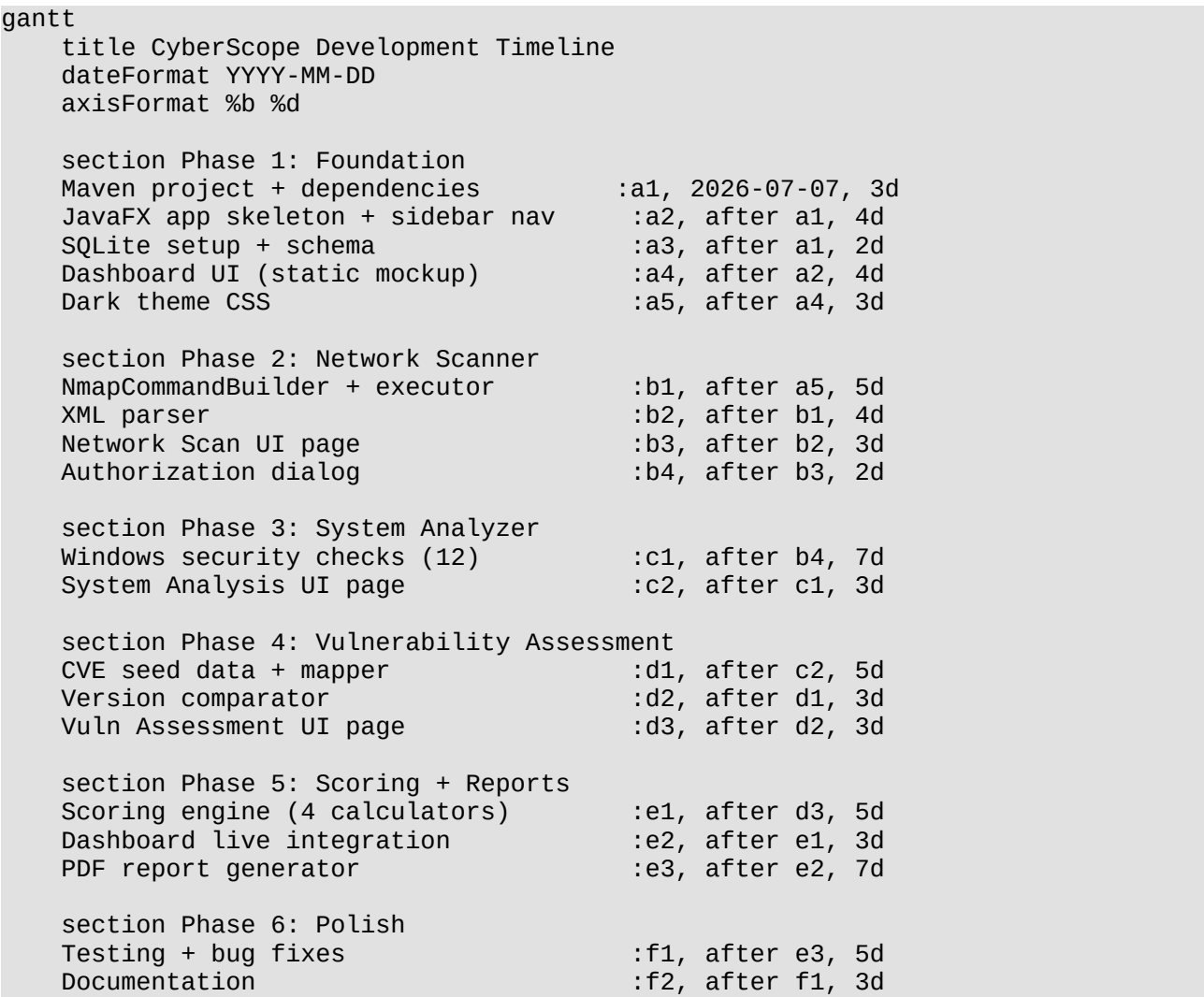
Category	Starting	Deductions	Score
Ports	100	6 regular ports × (−3) = −18, FTP (−8), RDP (−8)	66
Hardening	100	Updates old (−15), No lockout (−10)	75
Vulnerabilities	100	1 Critical (−15), 2 High (−10 each)	65
Configuration	100	FTP cleartext (−8), AutoRun (−5)	87

$$\text{Total} = (66 \times 0.20) + (75 \times 0.25) + (65 \times 0.35) + (87 \times 0.20) = 13.2 + 18.75 + 22.75 + 17.4 = \mathbf{72.1}$$

Risk Level: **Good** — Solid baseline, some areas need attention.

10. Development Timeline

10.1 Overview (10 Weeks)



10.2 Detailed Weekly Plan

Week	Phase	Deliverables	Learning Focus
1	Foundation	Maven project compiles, JavaFX window launches with sidebar, SQLite database created with schema	Maven lifecycle, JavaFX application structure, FXML basics
2	Foundation + UI	Dashboard page with static cards and placeholder charts, dark theme applied, page switching works	CSS styling in JavaFX, JavaFX layout panes, AtlantaFX components
3	Network Scanner	NmapCommandBuilder constructs valid commands,	ProcessBuilder API, Nmap command flags, XML file I/O

Week	Phase	Deliverables	Learning Focus
		NmapExecutor runs Nmap and captures XML output	
4	Network Scanner	NmapOutputParser converts XML to Java objects, Network Scan page displays hosts and ports in tables, authorization dialog works	Jackson XML parsing, JavaFX TableView, concurrent Task
5	System Analyzer	All 12 Windows checks implemented and tested, results stored in SQLite	Windows security commands (netsh, PowerShell, registry), DAO pattern
6	System Analyzer + UI	System Analysis page shows check results with pass/fail/warning indicators, results are interactive	JavaFX ListView with custom cells, conditional styling
7	Vulnerability Assessment	CVE seed data loaded (50+ entries), VulnerabilityMapper matches services to CVEs, Vuln page shows findings sorted by severity	CVE structure, CVSS scoring, semantic version comparison
8	Scoring Engine	All 4 category calculators working, overall score computed, Dashboard now shows live data from latest scan	Algorithm implementation, data aggregation, JavaFX chart binding
9	Report Generation	PDF reports generated with all sections (cover, summary, scores, findings, recommendations), export button on Reports page	Apache PDFBox API, document layout, chart-to-image rendering
10	Polish & Testing	All integration tests pass, error handling for edge cases, Settings page functional, README and ARCHITECTURE docs written	Testing patterns, documentation, code cleanup

10.3 Milestones

Milestone	Week	Validation
M1: App Launches	2	JavaFX app starts with dark theme, sidebar works, dashboard shows placeholder data
M2: First Scan	4	Can scan localhost or 192.168.x.x, see hosts and ports in table
M3: System Audited	6	Can run system analysis and see 12 pass/fail checks
M4: CVEs Identified	7	Discovered services are mapped to CVEs with severity levels
M5: Score Computed	8	Dashboard shows real security score from actual scan data
M6: Report Generated	9	Professional PDF can be exported and opened
M7: Project Complete	10	All features working, tested, and documented

11. Testing Strategy

11.1 Unit Testing (JUnit 5)

What to test: Pure logic with no external dependencies.

Component	Test Cases	Example
VersionComparator	Version comparison, range matching	<code>compare("2.4.41", "2.4.51") → negative</code>
PortScoreCalculator	Score with 0 ports, 1 dangerous port, 20+ ports	<code>calculate(ports with FTP) → deduction of 8</code>
HardeningScoreCalculator	All checks pass, all checks fail, mixed	<code>calculate(firewallOff, avOn) → 100 - 20 = 80</code>
VulnScoreCalculator	No CVEs, 1 critical, multiple mixed	<code>calculate(1 critical, 2 high) → 100 - 15 - 20 = 65</code>
NetworkValidator	Valid IPs, invalid IPs, CIDR ranges, injection attempts	<code>validate("192.168.1.1 ; rm -rf /") → false</code>
NmapCommandBuilder	All scan presets, custom targets	<code>build(STANDARD, "10.0.0.1") → correct flags</code>

11.2 Integration Testing

Test Scenario	Method
Nmap XML parsing	Use pre-captured Nmap XML files (no live Nmap needed) as test fixtures
Database operations	Use in-memory SQLite (<code>jdbc:sqlite::memory:</code>) for fast, isolated tests
CVE mapping	Insert test CVEs, insert test services, verify correct matches
Full scoring pipeline	Create a mock scan session with known data, verify final score matches hand calculation

11.3 Security Testing (of CyberScope itself)

Test	What It Validates
SQL injection in target input	Ensure user input is parameterized, never concatenated into SQL
Command injection in Nmap target	Ensure shell metacharacters (<code>;</code> , <code> </code> , <code>&</code>) are rejected by NetworkValidator
Nmap privilege handling	Ensure the app handles "requires root" errors gracefully
Database file permissions	Ensure the SQLite file is not world-readable

11.4 Manual / UI Testing

Test	Steps
Sidebar navigation	Click each menu item → verify correct page loads
Scan lifecycle	Start scan → verify progress bar → verify results appear → verify dashboard updates
Report generation	Run a scan → go to Reports → click Generate → verify PDF opens correctly
Error handling	Remove Nmap from PATH → start a scan → verify helpful error message appears
Resize behavior	Resize window to minimum → verify layout doesn't break

12. Security & Legal Considerations

12.1 Authorization Requirement

Before any network scan, CyberScope displays a mandatory consent dialog:

⚠ Authorization Required

Network scanning may only be performed on systems you own or have explicit written authorization to test.

Unauthorized scanning is illegal under:

- Information Technology Act, 2000 (India) - Section 43/66
- Computer Fraud and Abuse Act (USA)
- Computer Misuse Act 1990 (UK)

☐ I confirm I have authorization to scan the specified target(s)

[Cancel]

[I Acknowledge]

The "I Acknowledge" button is disabled until the checkbox is checked. This consent is logged in the scan session record.

12.2 Input Sanitization

All user-provided input (target IP, hostname) is validated before being passed to ProcessBuilder:

- **Whitelist approach:** Only allow characters [a-zA-Z0-9.\-/:]
- **Reject:** ;, |, &, `, \$, (,), {, }, <, >
- **Validate format:** Must match IPv4, CIDR, or hostname patterns

12.3 Principle of Least Privilege

- CyberScope requests admin/root privileges only when required (OS detection, some Nmap scan types)
- System hardening checks that don't require elevation run without it
- The application clearly indicates when elevated privileges are needed

13. Future Enhancements

Priority	Enhancement	Complexity	Value
v2.0	Linux system hardening analyzer	Medium	Cross-platform support
v2.0	Scan history with trend charts	Low	Shows security improvement over time
v2.0	NVD REST API	Medium	Real-time vulnerability

Priority	Enhancement	Complexity	Value
	integration (live CVE lookups)		data
v2.1	HTML report export	Low	Web-viewable reports
v2.1	Custom security check profiles	Medium	Tailored assessments
v3.0	Multi-host aggregate dashboard	High	Enterprise-scale view
v3.0	Compliance mapping (CIS, NIST)	High	Regulatory alignment
v3.0	REST API for automation	High	CI/CD pipeline integration
v3.0	SIEM export (Syslog/JSON)	Medium	SOC integration

14. References

#	Reference	Relevance
1	NIST SP 800-53 Security Controls	Security control framework that inspired the scoring categories
2	NIST National Vulnerability Database (NVD)	CVE data source and CVSS scoring methodology
3	MITRE CVE Program	Vulnerability identification standard
4	CVSS v3.1 Specification	Vulnerability severity scoring methodology
5	CIS Benchmarks	System hardening baselines used as reference for security checks
6	Nmap Reference Guide	Official Nmap documentation for scan types and output formats
7	Nmap XML Output Format	XML DTD specification for parsing Nmap results
8	OpenJFX Documentation	JavaFX framework reference
9	AtlantaFX Theme Library	Modern JavaFX theme components
10	Apache PDFBox	PDF generation library documentation
11	OWASP Testing Guide	Security testing methodology reference

#	Reference	Relevance
12	IT Act, 2000 (India)	Legal framework for authorized security testing in India

Document prepared as part of B.Tech Cybersecurity and Privacy program at Dehradun Institute of Technology (DIT).